CS152B Lab 1: Digital Design Project Laboratory

Katharine Archer (UID: 305831393)
Yashica Prasad (UID: 006090382)

April 16, 2026

1 Introduction

The objective of this lab is to design and test a custom 16-bit asynchronous ALU in purely structural Verilog. An ALU is a foundational digital circuit within a computer's CPU that performs mathematical and logical operations on binary data, depending on which control signals are asserted on the hardware block.
This ALU implementation can perform the following **signed** operations:

- Subtraction
- Addition
- Decrement
- Increment
- Invert
- Arithmetic Shift Left
- Arithmetic Shift Right
- Set on Less than or Equal

And the following **unsigned** operations:

- Bitwise OR
- Bitwise AND
- Logical Shift Left
- Logical Shift Right

1.1 Relevant Engineering Standards

Engineering Standards are documented, consensus-based technical specifications or procedures recognized as authoritative in a specific area of interest. Examples relevant to this lab include IEEE two's complement for the ALU's signed arithmetic, and IEEE 1364, the foundational standard for Verilog HDL itself.

1.2 Engineering Constraints

Engineering constraints are limitations or restrictions of a system. A relevant constraint of the system is the 16-bit width of the inputs, which informs the values that result in overflow for arithmetic operations.

2 Schematic and Code

Code lives on github: https://github.com/Katharine-a/digital_design/tree/main/alu

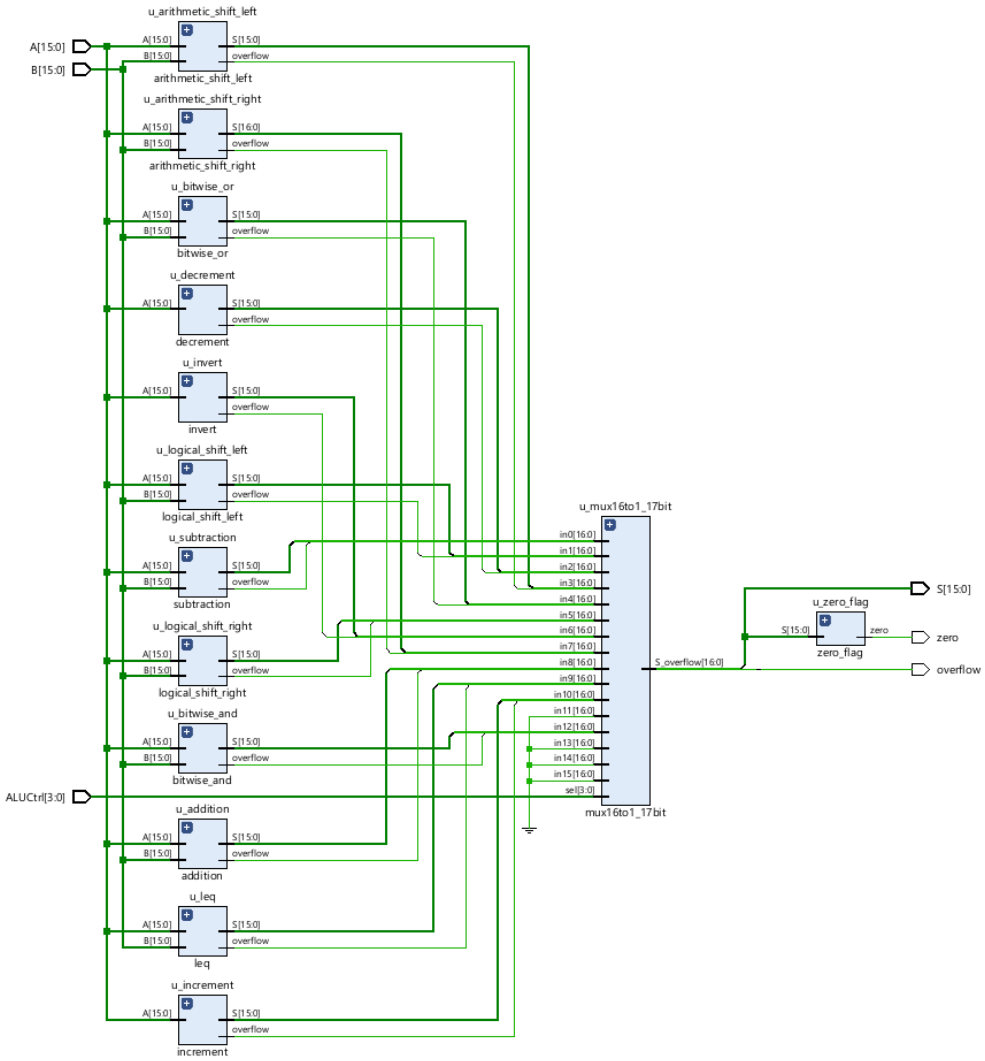

Figure 1: Top Level ALU Block Diagram

3 Description of Components

The ALU is of modular design and implemented in structural Verilog. Each operation and overflow result are computed and concatenated in parallel via individual modules. These outputs are multiplexed based on the ALU control signal, forming the S and overflow outputs. Finally, the zero flag is assigned.

3.1 Top-Level ALU Module

The `alu_top_module` serves as the central integration point for the design. It accepts two 16-bit inputs (`A[15:0]` and `B[15:0]`) and a 4-bit select signal (`ALUCtrl[3:0]`). Each operation module computes its result and the overflow flag in parallel, concatenates the results, feeds them into a 17-bit 16-to-1 multiplexer which selects the correct computation according to the ALU control bits, and assigns the S and overflow outputs of the module. The module lastly computes and assigns the zero status flag output.

3.2 Adder and Subtractor

The addition module combines a 16-bit ripple-carry-adder (`adder_16bit`) composed of 16 `full_adder` modules. The subtraction operation is performed using two's complement arithmetic by performing a bitwise inversion the second operand and passing `1'b1` into the carry-in bit of the adder module. Both modules delegate overflow detection to

the `adder_16bit` module, which computes it as the XOR of the carry into the most significant bit and the carry out of the most significant bit.

3.3 Increment and Decrement

The increment and decrement modules use `adder_16bit` to compute overflow, and add and subtract 1, respectively. This eliminates the duplication of gate level operations and maintains consistency for how status flags are handled.

3.4 Bitwise AND and OR

The bitwise AND and OR operations are implemented with gate primitives and a generate loop. Each module instantiates 16 (AND/OR) gates (one per bit). A `genvar` loop is used to replicate the logic across all bits, where each output `S[i]` is computed from `A[i]` and `B[i]`. Overflow is hard-coded to 1'b0.

3.5 Invert

The invert module performs a bitwise NOT on a 16-bit input using 16 `not` gates instantiated via a generate loop, and increments the resulting value. This operation (∼A+1) is functionally equivalent to multiplying the input value by -1. The output is a 17-bit signal, where the lower 16 bits are the inverted result and the final bit represents overflow. Overflow is detected for the case where the input is the minimum signed 16-bit value (`1000_0000_0000_0000`) by checking that the sign bit is high and all other bits are zero.

3.6 Shift Operations

The ALU supports both logical and arithmetic shifts:

1. `logical_shift_left` and `logical_shift_right` shift bits and fill vacated positions with zeros. In accordance with the lab specification, overflow flag is always zero for logical shifts.

2. `arithmetic_shift_left` and `arithmetic_shift_right` preserve the sign bit for signed operations.
   The `arithmetic_shift_left` uses `logical_shift_left` internally, since they are functionally equivalent. Overflow occurs if the sign bit of the result, S[15], is different from the original sign bit, A[15].

The shifter is implemented as a logarithmic barrel shifter, where the shift amount is decomposed bit-by-bit and applied in stages. At each stage, a multiplexer selects between the value shifted by the corresponding power-of-two offset and the passthrough value, conditioned on the respective bit of the shift amount, which achieves any arbitrary shift in O(log N) mux stages. An additional output-stage multiplexer handles the edge case where the shift amount exceeds 15: since any shift greater than or equal to 16 on a 16-bit value yields either all zeros (logical, arithmetic left) or a sign-flooded value (arithmetic right), the result is forced to that constant rather than the barrel shifter output.

3.7 Comparison Operation (leq)

The `leq` module performs a l̈ess than or equal toc̈omparison between the two inputs. This is implemented by analyzing the result of subtraction and evaluating the sign and zero conditions. $A < B$ is detected by XORing the sign bit of the result with the overflow flag. Equality is detected by the `zero_flag` submodule, and the two conditions are OR'd together to produce the final single-bit result, zero-extended to 16 bits.

3.8 Multiplexer Network

A 16-to-1, 17-bit multiplexer selects the concatenation of the final ALU output and overflow flag. This multiplexer is constructed hierarchically using 15, 17-bit, 2-to-1 multiplexers (using `mux2to1_1bit` and `mux16to1_17bit`). The selection is controlled by the 4-bit signal `sel[3:0]`, which determines which operation result is forwarded to the output.

3.9 Zero Flag

The `zero_flag` module detects when the ALU output is zero by wiring all result bits into a NOR gate.

4 Challenges

1. The 16 to 1 mux used to select the final output of the ALU is comprised of 15, 2 to 1 multiplexers arranged in a tree-like structure, with each but of the 4-bit op-code acting as the select bit for each layer of the tree. A simple testbench for the multiplexer (`mux_tb`) revealed that the outputs were not correctly correlated to the control bits asserted on the hardware block. The challenge of debugging the multiplxer was resolved through manual waveform inspection, which revealed that the op-code bits were passed in reverse order during initial development, causing the result of the reverse of the opcode to appear at the output. This issue was resolved by switching the order in which the op-code bits were passed in as select bits in the mux.

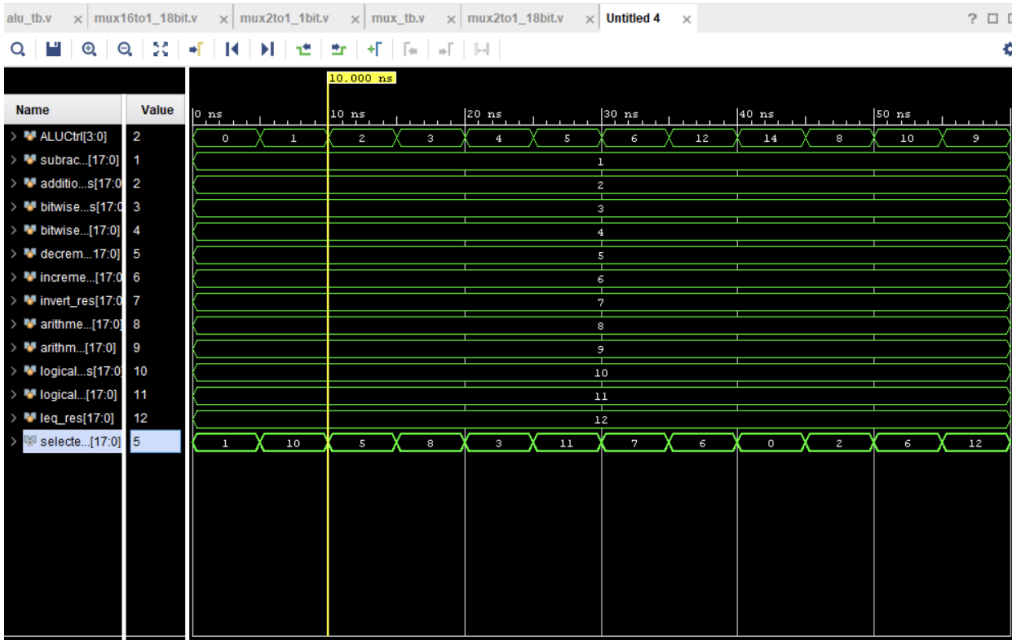

Figure 2: Waveform from a buggy 16 to 1 multiplexer

2. The challenge of writing a testbench for the ALU in a readable and self-checking manner led to the addition of an `assert_result` module which continuously assigns the passing or failing state of each testcase to the output wire `test_failed` by asserting the expected outputs against each set of test inputs. This addition significantly reduced the amount of time that otherwise would have been spent manually verifying the outputs of the waveforms.

```
module assert_result(
    input wire [15:0] S,
    input wire overflow,
    input wire zero,
    input wire [15:0] expected_S,
    input wire expected_zero,
    input wire expected_overflow,
    output wire test_failed
    );

    assign test_failed = (S !== expected_S) ||
                         (overflow !== expected_overflow) ||
                         (zero !== expected_zero);
endmodule // assert_result
```

3. We initially implemented the logic to select the specified operation using a singular switch-case statement. Later on, we realized that one of the constraints of structural Verilog was that control flow statements like switch-case or if else statements are not permitted. The challenge of selecting a single output from a larger grouping was resolved with multiplexers.

5 Questions

5.1 What is the difference between structural and behavioral Verilog? Provide an example of a structural and behavioral implementation of a multiplexer.

1. **Structural Verilog** describes combinational circuits in terms of its components and their connections. This paradigm characterizes the system at the gate level by explicitly defining how gates (and, or, not, nor, xor, etc.) and registers interconnect to form the the overall system; analogous to how a circuit is represented in a schematic diagram. This kind of code is limited to the use of wires, logic gates, and simple logic like loops.

   Multiplexer Implementation:

```
module mux2to1_1bit(
    input wire a,
    input wire b,
    input wire sel,
    output wire out
);
    wire sel_n, w1, w2;

    not (sel_n, sel);   // inverted select
    and (w1, a, sel_n); // input A selection
    and (w2, b, sel);   // input B selection
    or  (out, w1, w2);  // combine results
endmodule // mux2to1_1bit
```

2. **Behavioral Verilog** describes a circuit in terms of its functionality without detailing its internal structure. This paradigm models what a system does while abstracting away the lower-level details. Behavioral Verilog describes the circuit using a state machine with a set of sequential and combinational logic. It may use registers, complex logic (arithmetic, shifting, etc.), and complex operations.
   Multiplexer Implementation:

```
module mux2to1_1bit(
    input wire a,
    input wire b,
    input wire sel,
    output wire out
);
    assign out = sel ? a : b;
endmodule
```

5.2 What is the difference between an asynchronous and synchronous Multiplexer? Provide a brief explanation on how you could implement both using behavioral Verilog.

Synchronous multiplexers operate in accordance to a clock signal; their output changes at specific clock edges. The output of an asynchronous multiplexer changes immediately relative to input signals without waiting for a clock signal. In behavioral Verilog, a synchronous multiplexer can be implemented using an always block triggered by a clock edge, whereas an asynchronous multiplexer *could* be implemented with an always block with a sensitivity list of only the input signals.

5.3 What is the difference between a arithmetic and logical shifter?

Logical right shifters are zero-fill; they fill vacated bits with zeros. Arithmetic right shifters preserve the sign bit via sign extension of the most significant bit (MSB).
Example: `(0b1000_0000 >> 3)` produces

1. **Logical shift:** `0b0001_0000`
2. **Arithmetic shift:** `0b1111_0000`

Logical and arithmetic left shifters both fill the LSBs with zeros.
Example: `(0b1000_1000 << 3)` produces

1. **Logical shift:** `0b0100_0000`
2. **Arithmetic shift:** `0b0100_0000`

6 Testing

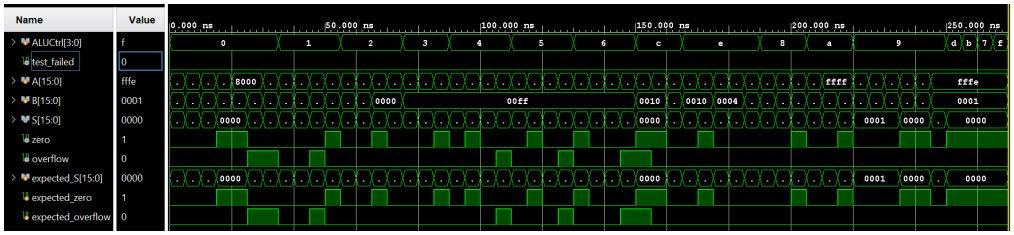

Figure 3: Waveform from Full ALU Testbench

6.1 ALU Testbench

Each ALU operation was verified using representative test cases covering normal operation, edge cases, and overflow conditions.

1. **Subtraction**
   - Positive result: $4 - 1 = 3$
   - Negative result: $6 - 10 = -4$
   - Negative operands: $(-32768) - (-32767) = -1$
   - Zero result (nominal and INT_MIN): $A = B \Rightarrow 0$
   - Overflow cases: $(-32768 - 1)$ and $(0 - (-32768))$

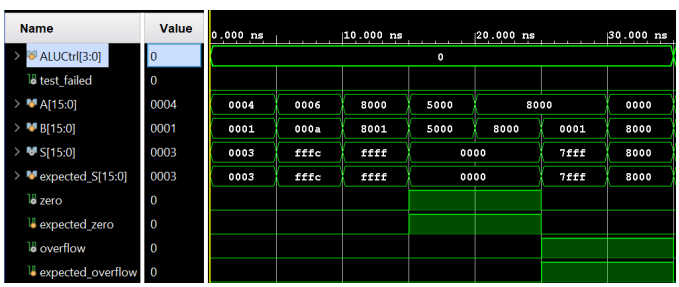

Figure 4: Subtraction Test Waveform

2. **Addition**
   - Positive sum: $10 + 5 = 15$
   - Negative sum: $(-5) + 2 = -3$
   - Overflow: $32767 + 1$
   - Zero result: sum equals 0

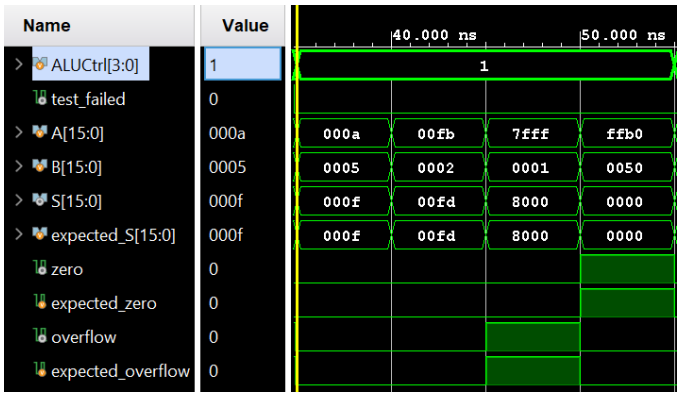

Figure 5: Addition Test Waveform

3. **Bitwise OR**

- Mixed bits: 0x00F0 — 0x0F00 = 0x0FF0
- Cross-bit test: 0x8001 — 0x0002 = 0x8003
- All zeros and all ones cases

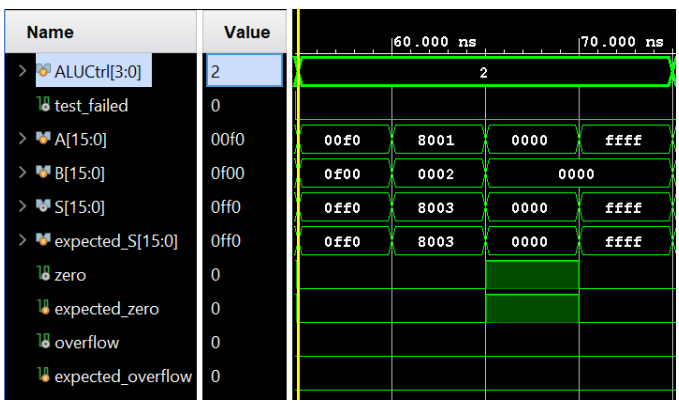

Figure 6: Bitwise OR Test Waveform

4. **Bitwise AND**

- Partial overlap: 0x0F0F & 0x00FF = 0x000F
- Non-zero result: 0x1234 & 0x00FF = 0x0034
- Zero result case

Figure 7: Bitwise AND Test Waveform

5. **Decrement**

   - $0 - 1 = -1$
   - $1 - 1 = 0$
   - Arbitrary value decrement
   - Overflow at minimum value

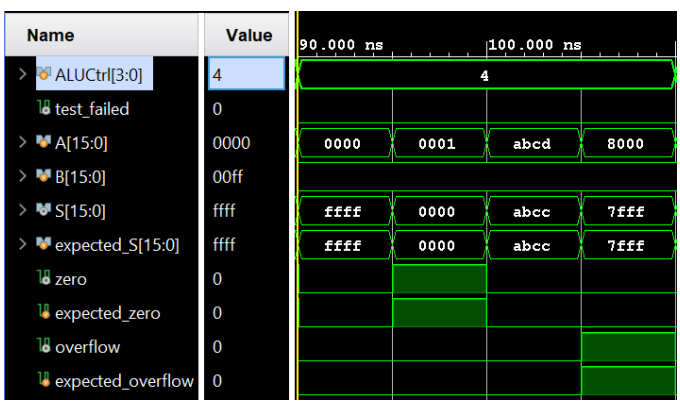

Figure 8: Decrement Test Waveform

6. **Increment**

   - $0 + 1 = 1$
   - $-1 + 1 = 0$
   - Arbitrary value increment
   - Overflow at maximum value

Figure 9: Increment Test Waveform

7. **Invert**

   - Inversion of 0 and -1
   - Arbitrary value inversion
   - Overflow at minimum signed value

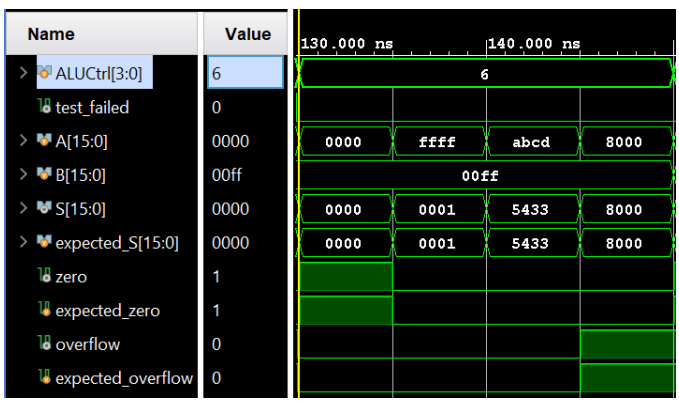

Figure 10: Invert Test Waveform

8. **Arithmetic Shift Left**

   - Large shifts ($\geq$ 16 bits)
   - Boundary shifts (15 bits)
   - Overflow detection cases

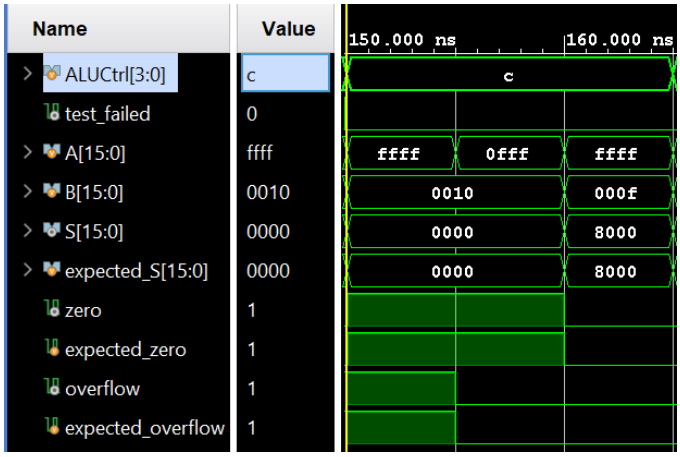

Figure 11: Arithmetic Shift Left Test Waveform

9. **Arithmetic Shift Right**

   - Sign-preserving shifts on negative values
   - Zero-fill behavior for positive values
   - Variable shift amounts

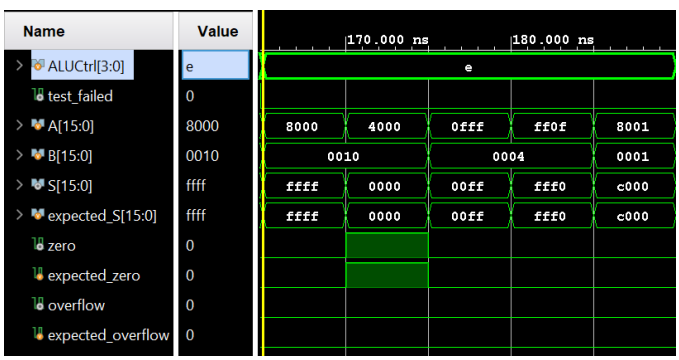

Figure 12: Arithmetic Shift Right Test Waveform

10. **Logical Shift Left**

   - Standard shifts
   - Maximum shift distances
   - Shift beyond input size

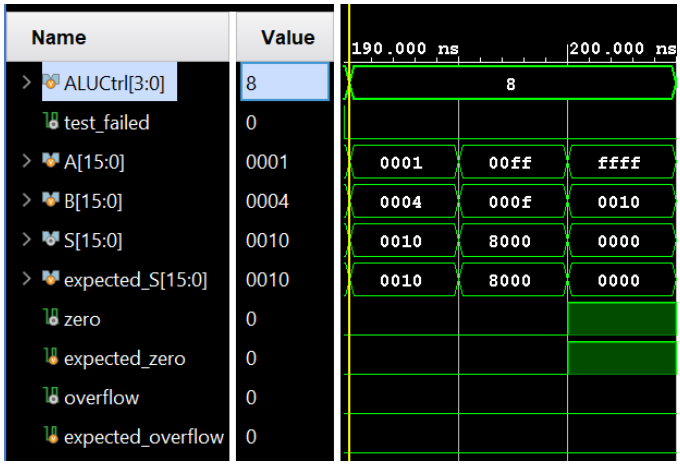

Figure 13: Logical Shift Left Test Waveform

11. **Logical Shift Right**

- Standard right shifts
- Large shift amounts
- Zero-fill verification

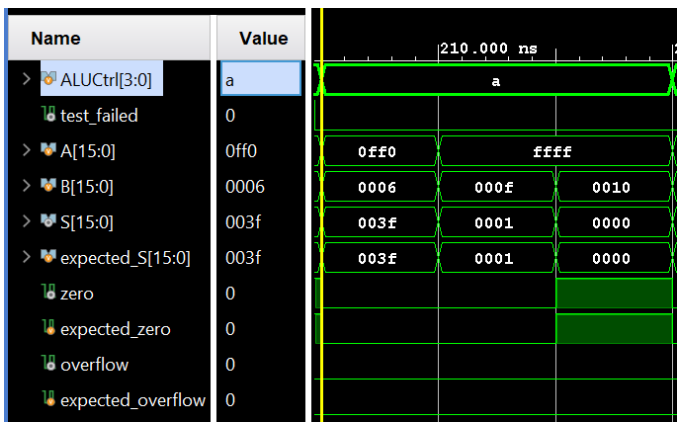

Figure 14: Logical Shift Right Test Waveform

12. **Set on Less Than or Equal**

- Equal values
- $A < B$ (positive and negative cases)
- $A > B$ (no set)
- Mixed sign comparisons

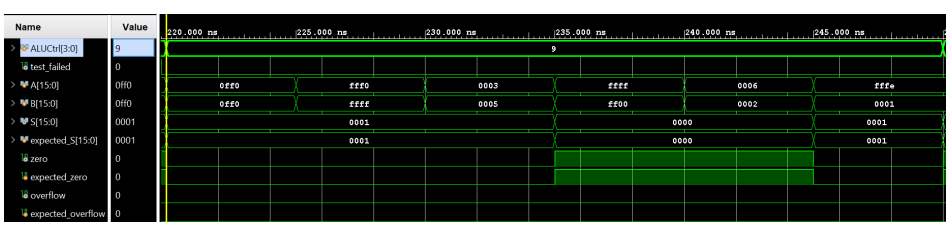

Figure 15: Set on Less Than or Equal Test Waveform

13. **Invalid Opcodes**

- Undefined control signals default to zero output

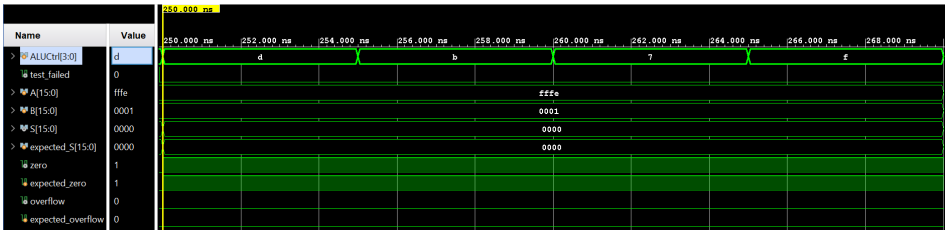

Figure 16: Invalid Op-codes Test Waveform